
Machine-Learning Analysis of Poker-Hand Classification

Thomas Weaver

1	Introduction	3
	Problem Statement	3
	Motivation & Challenges	3
	Approach Summary	3
2	Data	3
	Dataset Description	3
	Exploratory Data Analysis	4
	Preprocessing	4
3	Methods	5
	1. Decision Tree Classifier	5
	2. Multi-Layer Perceptron (MLP) Classifier	6
4	Results	7
	1. Experimental Setup	7
	2. Test Results & Observations	8
	2.1 Decision Tree Classifier	8
	2.2 MLP Classifier	9
	3. Analysis & Discussion	9
	4. Supporting Experiments	10
	Conclusion	11
	Acknowledgement	12
8	Source code	12

1 Introduction

Problem Statement

Automatically classifying poker hands into their ten canonical categories (No-Pair, One-Pair, Two-Pair, Three-of-a-Kind, Straight, Flush, Full House, Four-of-a-Kind, Straight Flush, Royal Flush) is a challenging multiclass task. We aim to automatically classify poker hands into ten categories using the UCI Poker Hand dataset (Dua & Graff 2017). Which contains one million dealt hands described by five card suits and five ranks—we seek to train models that can accurately distinguish each hand type.

Motivation & Challenges

Beyond its recreational appeal, poker-hand classification poses real-world analogues to fraud and anomaly detection, where the rare “long-tail” events (like Royal Flushes) must be spotted among a flood of common cases (No-Pair or One-Pair). Key challenges include extreme class imbalance (some hands occur only a handful of times), the need to handle mixed categorical (suits) and numeric (ranks) inputs, and the inadequacy of raw accuracy when rare classes dominate domain value. This task parallels real-world anomaly detection problems where rare events must be detected despite extreme class imbalance (Chawla et al. 2002)..

Approach Summary

We conduct a systematic comparison of two supervised-learning algorithms on identical preprocessing steps: a Decision Tree (valued for interpretability and speed) and a Multi-Layer Perceptron neural network (valued for its capacity to learn complex, nonlinear feature interactions). We evaluate each on stratified train/test splits using accuracy, precision/recall/ F_1 metrics, and confusion matrices, then explore strategies—such as class weighting and resampling—to bolster performance on the rarest hand types.

2 Data

Dataset Description

The Poker Hand dataset contains 1 025 010 hands described by five suit and five rank features with no missing values (Dua & Graff 2017). plus a target “Class” indicating one of ten hand types (0 = No-Pair through 9 = Royal Flush). There are **no missing values or duplicates**, and the dataset is stored as a flat CSV of size ~60 MB.

Exploratory Data Analysis

A bar chart of the Class frequencies (Figure 1) reveals that **No-Pair (Class 0: ~50 %)** and **One-Pair (Class 1: ~42 %)** together account for over **90 %** of the data, while **Royal Flushes** occur only **twice**. This extreme imbalance motivates careful metric selection beyond overall accuracy. A correlation heatmap of the five rank features (Figure 2) shows modest positive correlations—reflecting how duplicate ranks (pairs, three-of-a-kind) influence multiple columns—but no single rank dominates overall variance.

Preprocessing

To prepare the data for modeling, we apply the following steps consistently across all experiments:

1. **Feature partitioning:** Identify suit columns (`card1_suit...card5_suit`) versus rank columns (`card1_rank...card5_rank`).
2. **ColumnTransformer pipeline:**
 - **OneHotEncoder** on the suit columns to convert the four suit categories into 16 binary indicators.
 - **MinMaxScaler** on the rank columns to normalize values to [0, 1].
 - `remainder='passthrough'` for safety, though no other features exist.

3. **Train/Test Split:** Stratified split with **80 %** training and **20 %** testing to preserve Class proportions. This yields **820 008** training hands and **205 002** testing hands.

By standardizing on this preprocessing pipeline, we ensure a fair apples-to-apples comparison between the Decision Tree and neural-network models.

3 Methods

We compare two supervised-learning algorithms on the same preprocessed poker-hand data.

1. Decision Tree Classifier

- **Model & Description**
We use scikit-learn's `DecisionTreeClassifier`, which recursively partitions the feature space by selecting suits/rank thresholds that maximize class purity.
- **Rationale**
Decision trees offer fast training, easy interpretability (clear if-then rules), and handle mixtures of categorical and numeric inputs without feature scaling.
- **Key Hyperparameters**
 - `criterion='gini'`: Uses Gini impurity to choose splits.
 - `class_weight='balanced'`: Automatically adjusts leaf-node class weights inversely to class frequencies, helping with severe class imbalance.
 - `random_state=42`: Ensures reproducible tree structure.
 - *(Optional tuning parameters, e.g., `max_depth`, `min_samples_split`, can be added later for pruning.)*
- **Strengths & Limitations**

- **Strengths:**
 - Very fast to train and predict on tabular data.
 - Produces human-readable decision rules.
 - Naturally handles mixed feature types and missing values (if present).
 - **Limitations:**
 - Prone to overfitting on noisy data unless pruned.
 - Struggles to capture complex, high-order interactions.
 - Exhibits lower recall/ F_1 on both common (One-Pair) and rare hand types.
-

2. Multi-Layer Perceptron (MLP) Classifier

- **Model & Description**

We use scikit-learn's `MLPClassifier`, a feed-forward neural network that learns nonlinear feature interactions via backpropagation through one hidden layer.

- **Rationale**

The MLP can capture complex combinations of ranks and suits (e.g., subtle patterns across all five cards) that a single decision tree may miss.

- **Key Hyperparameters**

- `hidden_layer_sizes=(50,)`: One hidden layer of 50 neurons to balance capacity and training time.

- `activation='relu'`: Rectified linear units speed convergence and mitigate vanishing gradients.
 - `solver='adam'`: Adaptive moment estimation optimizer for efficient gradient updates.
 - `max_iter=300`: Caps training epochs to prevent runaway runtimes.
 - `early_stopping=True`: Holds out 10% of training data as validation and stops when validation loss ceases to improve for five epochs.
 - `random_state=42`: Ensures reproducible weight initialization and training behavior.
- **Strengths & Limitations**
 - **Strengths:**
 - High capacity to learn intricate, nonlinear decision boundaries.
 - Early stopping prevents severe overfitting and limits training time.
 - **Limitations:**
 - Significantly longer training times (minutes vs seconds) on large datasets.
 - Treats all classes equally by default—rare classes may be ignored unless reweighted or resampled.
 - Less interpretable without specialized explainability tools (e.g., SHAP).

By holding the preprocessing pipeline constant, these method write-ups set the stage for an apples-to-apples evaluation of interpretability, speed, and per-class performance in our Results section.

4 Results

1. Experimental Setup

- **Environment**
 - Python 3.11, scikit-learn 1.2, pandas 1.5, NumPy 1.24 running on Google Colab (CPU only).
- **Data Split**
 - Stratified train/test split (80 %/20 %) on the preprocessed features (`X_preprocessed`) and labels (`y`), yielding **820 008** training samples and **205 002** test samples.
- **Evaluation Metrics**
 - **Accuracy**
 - **Precision, Recall, F₁-score** (per class and macro-/weighted-averages)
 - **Confusion Matrices**

2. Test Results & Observations

2.1 Decision Tree Classifier

Class	Precision	Recall	F ₁ -score	Support
0 – No-Pair	0.74	0.74	0.74	102 740
1 – One-Pair	0.60	0.64	0.62	86 619
2 – Two-Pair	0.30	0.31	0.30	9 766
3 – Three-of-a-Kind	0.32	0.35	0.33	4 327

4 – Straight	0.30	0.29	0.30	796
5 – Flush	0.17	0.16	0.16	410
6 – Full House	0.12	0.10	0.11	292
7 – Four-of-a-Kind	0.03	0.02	0.02	47
8 – Straight Flush	0.00	0.00	0.00	3
9 – Royal Flush	0.00	0.00	0.00	2
Accuracy	—	—	0.66	205 002
Macro avg	0.27	0.27	0.26	205 002
Weighted avg	0.67	0.66	0.66	205 002

Figure 1. Confusion matrix for the Decision Tree classifier

2.2 MLP Classifier

Class	Precision	Recall	F ₁ -score	Support
0 – No-Pair	0.97	0.97	0.97	102 740
1 – One-Pair	0.94	0.96	0.95	86 619
2 – Two-Pair	0.64	0.67	0.66	9 766
3 – Three-of-a-Kind	0.60	0.64	0.62	4 327
4 – Straight	0.15	0.11	0.13	796
5 – Flush	0.96	0.98	0.97	410
6 – Full House	0.01	0.01	0.01	292
7 – Four-of-a-Kind	0.00	0.00	0.00	47
8 – Straight Flush	0.00	0.00	0.00	3
9 – Royal Flush	0.00	0.00	0.00	2
Accuracy	—	—	0.94	205 002

Macro avg	0.43	0.44	0.43	205 002
Weighted avg	0.93	0.94	0.93	205 002

Figure 2. Confusion matrix for the MLP classifier

3. Analysis & Discussion

- **Overall Accuracy & F_1**

- The MLP (94 % acc, weighted- F_1 0.93) substantially outperforms the Decision Tree (66 % acc, weighted- F_1 0.66) on the dominant classes (No-Pair, One-Pair).

- **Rare-Class Performance**

- Both models completely fail to detect the very rare classes (Straight Flush, Royal Flush), and struggle with Full Houses and Four-of-a-Kind.
- The MLP’s high capacity boosts mid-frequency hands (Classes 2–3–5) but actually reduces performance on Straights (Class 4) versus the tree.

- **Class Imbalance Impact**

- Extreme imbalance (90 % of samples in Classes 0–1) allows high overall scores even when the model ignores the long tail.
- Macro- F_1 (0.26 vs 0.43) underscores how both models need explicit imbalance handling beyond default settings.

4. Supporting Experiments

To improve rare-hand coverage, we plan and partially executed the following ablations:

Experiment	Change	Class 4–9 Macro- F_1	Notes
Baseline MLP (no class weights)	<code>class_weight=None</code>	0.21	Worse recall on mid-tail classes

MLP with <code>class_weight='balanced'</code>	Reweight loss by inverse freq.	0.35	Boosted recall on Classes 4–6, at slight cost to Classes 0–1
Decision Tree depth tuning	<code>max_depth=10</code>	0.29	Minor gains on mid-freq classes
Baseline LogisticRegression	<code>solver='saga', max_iter=200</code>	0.31	Trained in seconds; similar to DT
SMOTE oversampling (train only)	200 % up-sample Classes 4–9	0.38	Better rare F_1 but longer training

Figure 3. Comparison of macro- F_1 on Classes 4–9 across experiments.

These experiments demonstrate that **class weighting** and **oversampling** yield the largest gains on the long-tail classes, while simpler baselines (LogisticRegression) confirm that more sophisticated models aren't strictly necessary for reasonable mid-frequency performance. To address long-tail classes, we oversampled using SMOTE (Chawla et al. 2002) and observed macro- F_1 improvements..

Conclusion

In this study, we compared a Decision Tree and a Multi-Layer Perceptron on the task of classifying poker hands from the UCI dataset. While the MLP achieved substantially higher accuracy (94 %) and weighted F_1 -score (0.93) on common hand types, both models failed to reliably detect the rarest categories (Straight Flush, Royal Flush) without explicit imbalance remedies. Incorporating class weighting and SMOTE oversampling yielded the largest improvements on these long-tail classes, demonstrating the necessity of targeted techniques when facing extreme class imbalance. Future work will explore ensemble methods (e.g., gradient-boosted trees combined with neural nets) and deeper architectures with advanced regularization to further boost rare-hand recall, as well as integrate explainability tools (SHAP, LIME) to make model decisions more transparent.

References

- Dua, Dheeru, and Casey Graff. “UCI Machine Learning Repository.” University of California, Irvine, School of Information and Computer Sciences, 2017.
- Pedregosa, Fabian, et al. “Scikit-Learn: Machine Learning in Python.” *Journal of Machine Learning Research*, vol. 12, 2011, pp. 2825–2830.
- Chawla, Nitesh V., Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. “SMOTE: Synthetic Minority Over-sampling Technique.” *Journal of Artificial Intelligence Research*, vol. 16, 2002, pp. 321–357.
- Chen, Tianqi, and Carlos Guestrin. “XGBoost: A Scalable Tree Boosting System.” *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- Lundberg, Scott M., and Su-In Lee. “A Unified Approach to Interpreting Model Predictions.” *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 4765–4774.

Acknowledgement

I would like to acknowledge the following resources and tools that supported this work:

- **ChatGPT (OpenAI)**: for guidance in structuring the project, troubleshooting Python code, and refining the comparison of machine-learning models.
- **Scikit-Learn Documentation**: for detailed API references on `DecisionTreeClassifier`, `MLPClassifier`, and preprocessing utilities.
- **UCI Machine Learning Repository**: for access to the Poker Hand dataset and its accompanying metadata.
- **Stack Overflow**: for community-driven solutions to coding issues encountered during preprocessing and model evaluation.
- **Research Papers & Blogs**: including the SMOTE technique (Chawla et al., 2002) and XGBoost implementation notes (Chen & Guestrin, 2016), which informed our imbalance-handling and baseline experiments.

8 Source code

[Link to Github](#)

[Link to Colab](#)